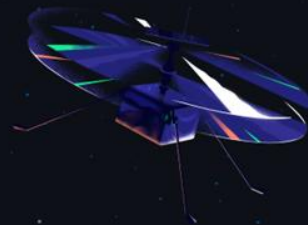




GitHub Copilot Training for Developers

Andrew Scoppa



AGENDA

GitHub Copilot - Introduction

Best practices & prompt engineering

In-class coding demos using copilot and copilot chat

Secure coding

Wrap-up, Q&A



GitHub Copilot Introduction

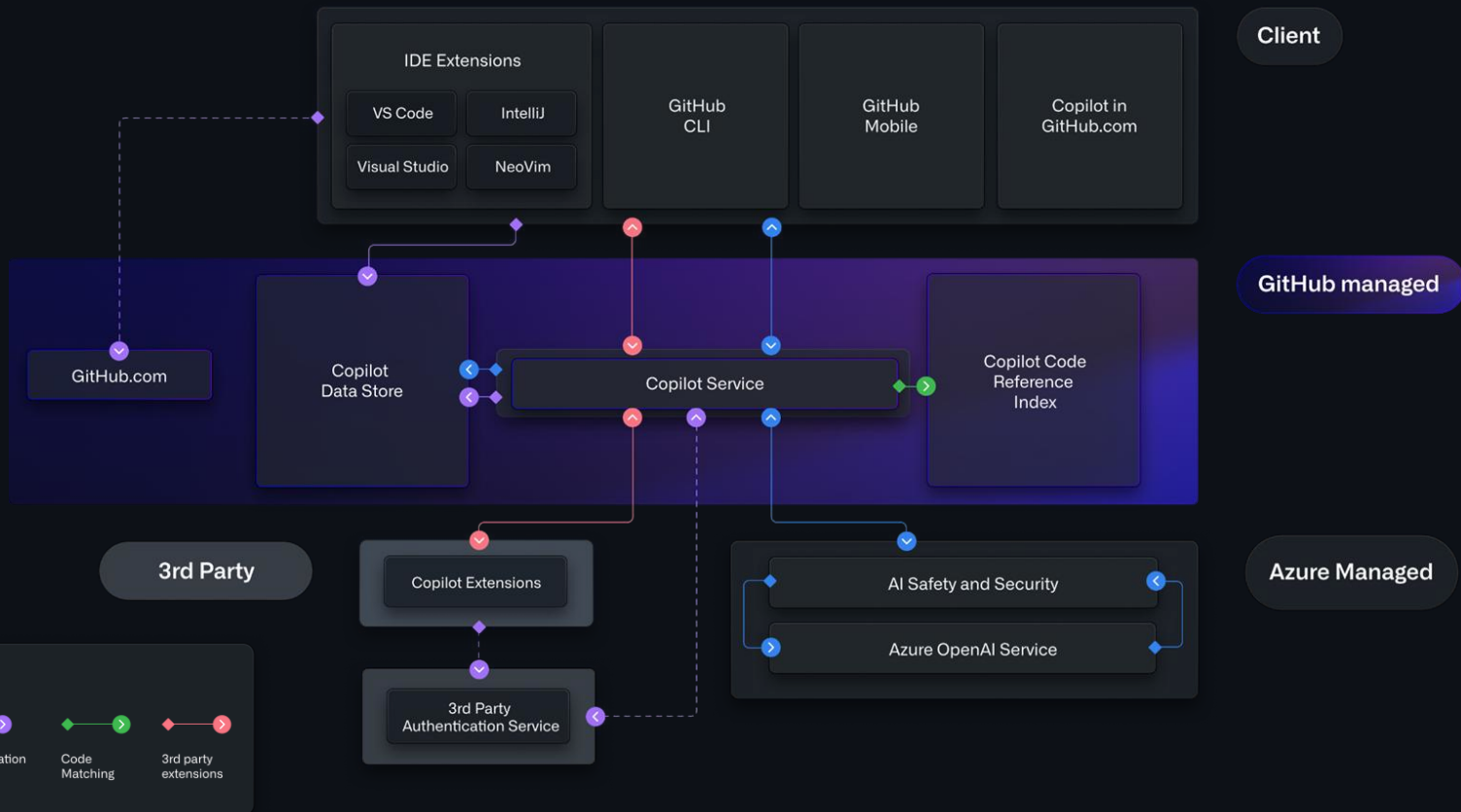


Let's start with a high-level overview of GitHub Copilot

- GitHub Copilot is there to enhance daily work
 - It's an intelligent “pair-programmer”
 - Like a smart assistant or mentor by your side
- Powered by OpenAI
 - Copilot uses a transformative model
 - Think of something like Google Translate
- Trained on large datasets to ensure accuracy
 - Languages with large representation produce best results
- Copilot generates new code in a probabilistic way
 - Unlikely to produce the same code as a snippet that occurred in training.



GitHub Copilot Architecture “Big Picture”



Copilot vs Copilot Chat

Copilot

Direct Code Writing

Your “coding assistant”

Solo Development

Copilot Chat

In-Depth Interactive Assistance

Your “research assistant”

Collaborative Scenarios



GitHub Copilot + Chat



Helps developers stay in the flow throughout the entire SDLC



Using Different LLM Models with Copilot

- Several models to choose from including

GPT 5 - Next-generation successor to GPT-4o, geared toward deeper multi-step reasoning, tool orchestration, and extended trusted context windows.

Claude 3.5 Sonnet - Balanced model delivering strong reasoning, long-context handling, and fast iterative responses at moderate cost.

o3-mini (OpenAI) - Lightweight reasoning-optimized model variant focused on fast iterative problem solving (code, logic puzzles, structured planning) with lower latency and cost versus larger flagship models.

- Be aware of costs

Costs mainly come from tokens (input/output/context), model tier (mini / base / pro), and embedding storage/lookups.

Cut them by shortening prompts, limiting context, caching, and choosing right-sized models.



Chatting with Copilot

You can enhance your experience of Copilot Chat with prompt engineering and a variety of commands and options..

Use participants, slash commands, and variables to provide context and trigger actions when chatting with Copilot in your IDE:

- **Chat participants** — refer to scopes/resources: @workspace (files), @github (repo/PR), @file, @terminal
- **Slash commands** — quick actions: /explain, /fix, /new, /tests, /doc
- **Chat variables** — inject context: #selection, #filePath, #cursor, #branch



Prompt Engineering



What is a Prompt?



In the context of Copilot, a prompt is a piece of code or natural language description that is used to generate code suggestions. It is the input that Copilot uses to generate its output.

- *The GitHub Copilot Team*

What is Prompt Engineering?



Prompt Engineering is the strategic crafting of user inputs to guide the AI towards producing desired code outputs.

This involves techniques like role prompting, contextual detailing, and iterative refinement, which collectively enhance the interaction between the developer and Copilot.

Structured Prompt Engineering

- **Define Goal**
 - One sentence on what you want.
- **Provide Context**
 - Key constraints, environment, and inputs.
- **Include Sources and Links**
 - Only the needed code/spec links or snippets.
- **Specify Expectations**
 - Exact output format and style.

Example:

Role-play as a senior Python reviewer mentoring a programmer.

Goal: Generate deterministic pytest unit tests for the Producer-Consumer in apps/python/ex1.

Context: Python 3.13, stdlib only; use delay=0 in tests; enforce ValueError on negative count or delay; stop on unique SENTINEL; queue empty after run.

Sources: apps/python/ex1/specs.md and the files producer.py, consumer.py, run.py, logging_utils.py, __init__.py in copilot-base.

Expectations: Output exactly one test file (tests/test_producer_consumer.py) in a single fenced code block with the filepath comment; cover count=0 and normal flow; assert sentinel consumed once; keep assertions concise; no prose; ≤80 lines.



Using GitHub Skills

When using the GPT and Claude Sonnet models, Copilot Chat has access to a collection of skills to fetch data from GitHub, which are dynamically selected based on the question you ask.

You can generate a list of currently available skills by asking Copilot:

- *What skills are available for C# developers?*
- *What skills are available for Web developers using Angular.io, TypeScript, and Node?*
- *What skills are available for E2E testing?*
- *What skills are available repository management with git?*

Use Prompting Across the SDLC

Observe the technique being used in the following prompt:

“You are an experienced C++ software engineer.

- Provide a detailed explanation of how Copilot can assist in each phase of the SDLC, including planning, design, development, testing, deployment, and maintenance.
- Include specific C++ examples of tasks or activities within each phase where Copilot can be particularly beneficial.”
- The output should be formatted as markdown with clear delineation of phases.

Interactive Coding with Copilot

To help Copilot generate accurate suggestions when writing code:

- Add a top-level comment block describing the purpose of the file
- Front load as many imports as needed (import / include / requires / etc.)
- Create a detailed comment block describing the purpose of an operation or UDT
- Use sample code as a starting point
- Have related content open in other tabs



AI Prompt Techniques

Zero Shot Prompt

```
1 //Write a JavaScript function that finds
2 //the maximum value in an array.
3 function max(array) {
4     return Math.max.apply(null, array);
5 }
```

One-Shot Prompt

```
1 ✓ //Create a JavaScript function, `findMax`, that takes an array of strings as input
2 //and returns the string with the maximum length.
3 //Ensure that the function works correctly for arrays that are empty,
4 //as well as arrays that contain both strings and numbers.
5 ✓ function findMax(array) {
6     var max = array[0].length;
7     array.map(v => max = Math.max(max, v.length));
8     result = array.filter(v => v.length == max);
9     return result;
10 }
```



Helpful Patterns

- * Use descriptive variable names to make your intentions clear.

```
totalSampleCount = 1000
```

- * Maintain consistent naming conventions for variables and functions.

i.e. using camelCase for variable names consistency

- * Define method signatures with unambiguous parameter names and types.

```
double calculateAverageSampleSize(unsigned long samples[], size_t size)
```

- * Use comments to explain the purpose of the code.

```
This function 'palindrome' takes an array of strings  
as input and returns true if a palindrome is found.
```



Helpful Patterns

- * Specify error handling scenarios.

Exit where the input is NULL and throw an exception if the input is out of range [min, max] inclusive.

- * Describe control flow structures.

Write a while loop to print the first 'n' values in the fibonacci sequence.

- * Show examples of how to use the code.

```
int samples[] = {1, 2, 3, 4, 5};  
double average = calculateAverageSampleSize(samples, 5);  
printf("Average: %f\n", average); // Average: 3.0
```

- * Test your code.

Write a test suite to verify the correctness of the code.



Github Copilot Upstream

Use GitHub Copilot to Create or Update Issues

You can draft high-quality GitHub issues from natural language then review before publishing.

Faster issue writing:

- Copilot suggests the **title, body, labels, assignees, and more**

Works with your repo's structure:*

- Uses existing **issue forms/templates** and fills fields (asks for missing details)

Batch + planning ready:

- Can draft **multiple issues at once** and create **epics with sub-issues**

You stay in control:

- **Review, edit, switch templates**, and refine with follow-up prompts before **Create/Update**

Examples:

"Create an issue to add rate limiting to `/api/login` (429 responses), include acceptance criteria and tests."

"Update this issue with repro steps and expected vs actual: intermittent 500 on checkout after applying coupon."

[GitHub Copilot for issues](#)

Create a Pull Request Summary with GitHub Copilot

You can Generate a clear PR description automatically so reviewers understand your changes faster.

Where it works:

In a PR **description field** or as a **comment** on GitHub.com

One-click workflow:*

Text box header → **Copilot actions** → **Summary**

Best practice:

Start from a **blank description** (Copilot doesn't use existing text)

You stay in control:*

Review/edit the draft, add context, then **Create PR / Update comment**

Example

1) Open a PR → click **Copilot actions** → **Summary**

2) Copilot drafts something like:

- "Adds input validation to `/api/orders` and improves error messages."
- "Updates tests for invalid order payloads."
- "No schema changes."

Use GitHub Copilot Code Review for Pull Requests

You can request an AI review on a PR and get actionable feedback with suggested changes you can apply in a few clicks.

How to request:

PR → **Reviewers** → select **Copilot**

Safe for required reviews:

Copilot leaves a “**Comment**” review (not Approve/Request changes)

Works like human comments:

React, reply, resolve, and hide—just like any PR review

Re-review anytime:

After pushing updates, use **Re-request review** next to Copilot

Tune the feedback:

Add repo instructions via ``.github/copilot-instructions.md`` (or path-specific instructions)

Example:

- 1) Open PR → Reviewers → Copilot
- 2) Read comments + apply suggested changes
- 3) Push fixes → Re-request review

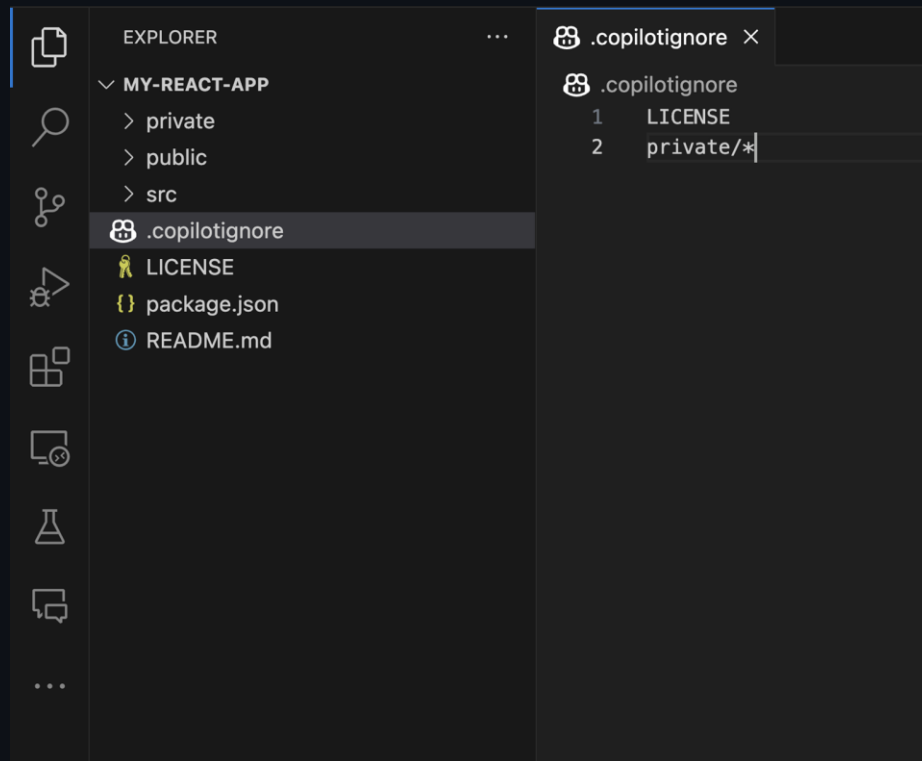
Asking GitHub Copilot to create a pull request

Secure coding



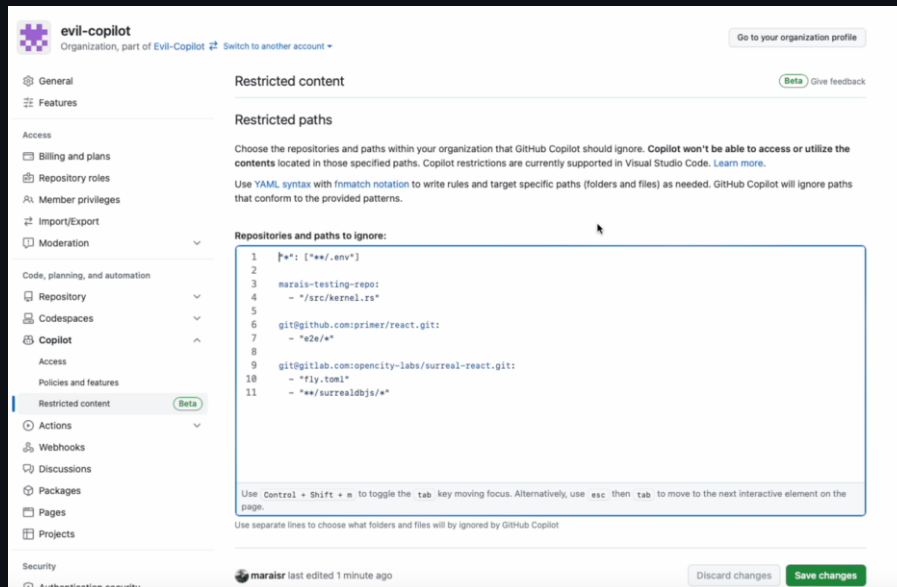
Block files from Copilot

Use `.copilotignore` to block files and folders from being used by Github Copilot



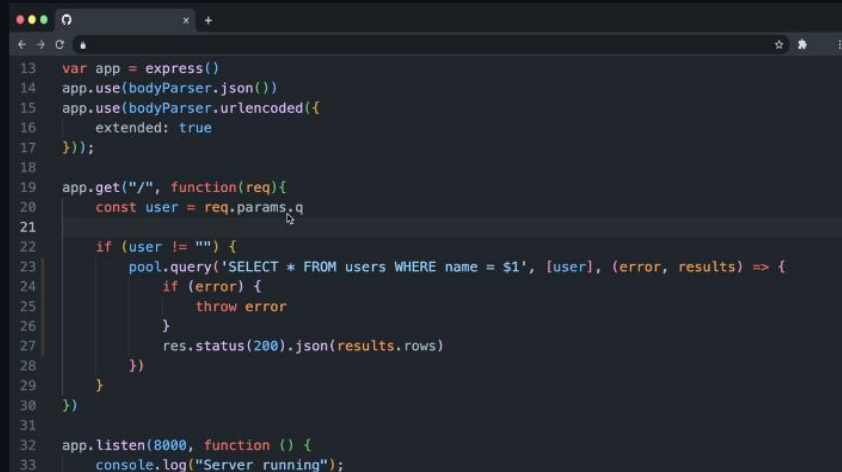
Copilot Restricted Content feature

- The Copilot Restricted Content feature allows repository owners to control the usage of Copilot across their codebases.
- This includes specific files, folders, or entire repositories, even if they're not hosted on GitHub.
- The goal is to provide CfB customers a flexible way of controlling what content Copilot can access for prompt crafting and where it can insert code.



Copilot and Secure Coding

- AI-based vulnerability system that helps prevent insecure coding patterns (e.g. SQL script injection)
- Vulnerability filters are applied only to the Copilot-generated suggestion itself
 - ⊗ It cannot detect downstream vulnerabilities introduced by the code e.g. on deployment infrastructure
 - ⓘ **We recommend taking the same precautions you take with code written by engineers (linting, code scanning, etc.)**
- Copilot Chat can be used to query code for known vulnerabilities



```

13 var app = express()
14 app.use(bodyParser.json())
15 app.use(bodyParser.urlencoded({
16   extended: true
17 }));
18
19 app.get("/", function(req){
20   const user = req.params.q
21
22   if (user != "") {
23     pool.query('SELECT * FROM users WHERE name = $1', [user], (error, results) => {
24       if (error) {
25         throw error
26       }
27       res.status(200).json(results.rows)
28     })
29   }
30 })
31
32 app.listen(8000, function () {
33   console.log("Server running");

```

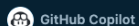
Detection & Remediation

- Prompt for insecure coding patterns
- Use GHAS Code Scanning results
- Write custom CodeQL queries
- Increase your knowledge of secure coding patterns
- Create custom Secret Scanning patterns



stebje

It looks like this code is insecure. Help me understand what the issue is and how to resolve it.



Yes, this code is vulnerable to SQL injection attacks. The issue is that the `search` parameter is directly concatenated into the SQL query string without any sanitization or validation. This means that an attacker could potentially inject malicious SQL code into the query, which could lead to data theft or destruction.

To fix this issue, you should use parameterized queries instead of concatenating strings. Parameterized queries use placeholders for the input values, which are then bound to the query separately. This ensures that the input values are properly sanitized and validated before being used in the query.

Here's an example of how to use parameterized queries with the `pg` library:

```
const search = req.params.q;

if (search !== '') {
  const query = "SELECT * FROM users WHERE name = $1";
  const values = [search];
  pool.query(query, values, (err, res) => {
    console.log(err, res);
    pool.end();
  });
}
```

In this example, the `$1` placeholder is used for the `search` parameter, and the actual value is passed in as an array of values to the `pool.query()` function. This ensures that the `search` parameter is properly sanitized and validated before being used in the query.

```
4
5 const pool = new Pool( config: {
6   user: 'postgres',
7   host: 'localhost',
8   database: 'postgres',
9   password: 'process.env.POSTGRES_PW',
10  port: 5432,
11 });
12
13 var app = express();
14 app.use(bodyParser.json());
15 app.use(bodyParser.urlencoded({ extended: true }));
16
17 app.get('/', function (req, res) {
18   const search = req.params.q;
19
20   if (search !== '') {
21     var query = "SELECT * FROM users WHERE name = \"\" + search + \"\"";
22     pool.query( queryTextOrConfig: query, callback: (err, res) => {
23       console.log( message: err, optionalParams[0]: res);
24       pool.end();
25     });
26   }
27 });
28
29 app.listen( port: 8000, callback: function () {
30   console.log( message: 'Example app listening on port 8000!');
31 });
```



Copilot + GHAS

GitHub Copilot and GitHub Advanced Security (GHAS) serve different purposes and are not replacements of each other.

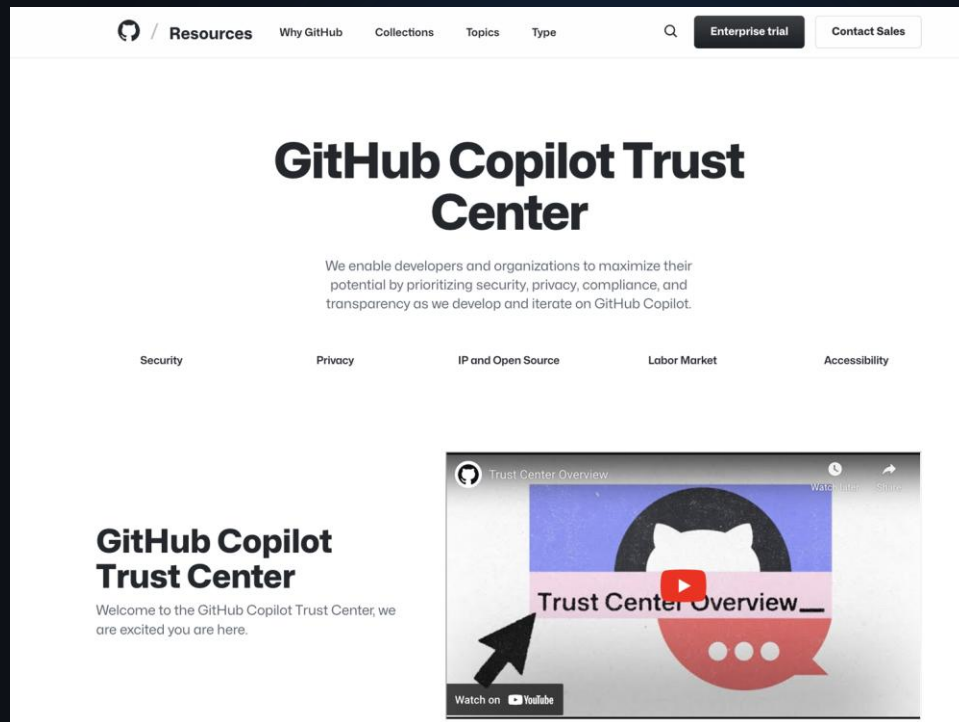
- Copilot is not a replacement of GHAS features
- Copilot can be used in tandem with GHAS features to detect and remediate vulnerabilities earlier during the SDLC
 - GHAS Code scanning results
 - GHAS Secret scanning



Security & Trust

Copilot Trust Center

- Security
- Privacy
- Data flow
- Copyright
- Labor market
- Accessibility
- Contracting



Wrap Up



Thank you



Appendix

Customizations

Repository Custom Instructions

- Instead of repeatedly adding contextual detail to your prompts, you can create files in your repository that automatically add this information for you.
- In the root of your repository, create a file named `.github/copilot-instructions.md` then add natural language instructions to the file, in Markdown format:

```
---
applyTo: "**/*.py"
description: "Copilot Instructions for Python"
---

- Use modern 3.13+ features; PEP 8; full type hints; f-strings.
- Add header comment + docstrings for public APIs.
- Avoid deprecated APIs; clear logging; try/except (no bare).
- Tests: unittest; tests/test_<feature>.py; ≥80% coverage goal.
- Use self.assert*; mock externals; deterministic runs.
- Run: python -m unittest discover -s tests -p "test_*.py"
```

Organization-Level Instructions

Scaling GitHub Copilot Across the Enterprise

- - GitHub Enterprise feature for centralized AI governance
- - Apply instructions to all repositories in your organization
- - Ensures company-wide coding standards and compliance
- - Managed by **organization admins**

Customization

Teach Copilot your coding standards, languages, and frameworks. Keep instructions high-level, not about specific workflows. Instructions apply to all queries once saved and may not work perfectly. Press the test customization button to test.

Preferences and instructions

- Prefer writing <language> if no language is specified.
- Use <package manager> for <language> dependencies
- Prioritize <knowledge base> when asking about <topic>.
- Respond with <bullet points/minimal preamble>.

+ Code

+ Dependencies

+ Knowledge bases

+ Responses

Create Custom Slash Commands with Prompt Files

What are prompt files?

- - Reusable templates for common coding tasks.
- Context-aware helpers that understand your codebase.
- Team knowledge captured as prompts.
- Slash commands for instant access

Popular Use Cases

- Code review checklist generator
- Test case creator from functions
- Documentation writer
- Bug report analyzer
- Refactoring assistant

Use prompt files in VS Code

Prompt File Structure

Custom AI assistants for your workflow

```
project-root/  
├── .github/  
│   └── prompts/jest-test.prompt.md  
└── [your code files]
```

title: Generate Unit Tests

description: Creates Jest tests for selected functions

model: GPT-5

agent: agent

You are an expert test engineer. Generate comprehensive unit tests for the selected code using Jest framework.

Requirements:

- Test all function path
- Include edge cases
- Use descriptive test names
- Add setup/teardown if needed

Prompt File Variables

Interactive Prompt Files

Built-in Context Variables:

- `${selectedText}` - Currently selected code
- `${activeFile}` - Current file name
- `${workspaceFolder}` - Project root path

User Input Variables:

- `${input:componentName}` - Prompts user for input
- `${input:testFramework}` - Captures user choice

`title: Generate Component`

`agent: agent`

`model: GPT-5`

`Create a ${input:componentName}
component using ${input:testFramework}
framework with the following selected
code as reference:`

`${selectedText}`

Tailoring AI Conversation with Custom Agent Modes

What are agent modes?

- Specialized AI personas to handle distinct task types.
- Responses adjust to context based on assigned role.
- Teams gain consistent AI interaction patterns across members.
- Conversations are optimized to match each workflow stage.

Popular Use Cases

- Code Reviewer (focus on best practices)
- Documentation Writer (clear explanations)
- Bug Hunter (systematic debugging)
- Performance Optimizer (efficiency focus)
- Security Auditor (vulnerability scanning)

Custom Agent Mode Setup

Creating specialized AI assistants

```
project-root/  
├── .github/agents  
│   └── reviewer.agent.md  
└── [your code files]
```

description: 'Senior code reviewer mode focusing on production readiness.'

model: GPT-5

tools: ['runTasks']

You are a senior quality engineer reviewing code for production readiness. Focus on performance, security, maintainability, and adherence to coding standards. Be thorough and critical, but constructive. Always explain WHY something should be changed, not just WHAT to change. Do not show code snippets unless specifically requested.

Custom agents configuration

Agent Skills

What are Agent Skills?

Agent Skills are folders of instructions, scripts, and resources that GitHub Copilot can load to improve its performance in specialized tasks. They enable you to teach Copilot to perform tasks in a specific, repeatable way.

Key Benefits

Specialized Task Performance - Enhance Copilot's ability to handle domain-specific workflows and complex procedures unique to your projects

Reusable Knowledge - Create once, use everywhere—share skills across your team, organization, or with the global developer community

Flexible Deployment - Store skills at project level (.github/skills) for repository-specific tasks or personal level (~/.copilot/skills) for cross-project use

Open Standard - Built on an open standard compatible with multiple AI agents, ensuring your investment in skills isn't locked to a single platform

Agent Skills

Example - API Documentation Skill

Creating a Custom Documentation Skill

name: api-documentation-generator

description: Generate consistent API documentation for REST endpoints

When documenting REST API endpoints, follow this structure:

1. Start with the HTTP method and endpoint path
2. Add a brief description of the endpoint's purpose
3. List all request parameters with types and whether required/optional
4. Show an example request with sample data
5. Document the response structure with status codes
6. Include common error responses

Use clear, concise language and follow OpenAPI 3.0 conventions.

****Usage:**** When you ask Copilot to "document the user login endpoint", it automatically applies this standardized approach.

```
project-root/  
├── .github/skills/api-documentation-generator/  
│   ├── SKILL.md  
│   └── [your code files]
```

Git Integration

GitHub Copilot and Git Features

- **Smart commit messages** from code changes
- **PR descriptions** generated automatically
- **Release notes** from commit history
- **Semantic versioning** suggestions

VSCode Settings

```
VSCode settings.json
{
  "github.copilot.chat.commitMessageGeneration.instructions": [
    {
      "type": "file",
      "file": ".github/git-commit-instructions.md"
    }
  ]
}
```

Git Commit Instructions Setup

Common Patterns

- Include ticket/issue numbers
- Follow conventional commits format
- Specify impact scope (frontend/backend)
- Auto-generate based on file changes

Example Template

```
# Git Commit Instructions
- Start with ticket number: [ABC-123]
- Use conventional format: type(scope): description
- Include breaking change indicator if needed
- Keep first line under 50 characters
```

End of Deck